

Week _02:

Task 1 :Theory and Research

1. OWASP Top 10 Vulnerabilities

[SQL Injection ,Cross-Site Scripting, Cross-Site Request Forgery, Security Misconfiguration, Broken Authentication & Session Management, Sensitive Data Exposure, Insecure Deserialization, Using Components with Known Vulnerabilities, Insufficient Logging & Monitoring,XML External Entity (XXE)]

2. Explore website security measures i.e. how websites secure themselves (CAPTCHA, HTTPS, Content Secure Policy etc.)

Task 1 : The OWASP top 10 vulnerabilities are following :

1. SQL Injection:

- Description: SQL Injection occurs when an attacker can insert or manipulate SQL queries within the input fields of an application. This can lead to unauthorized access to the database, data leakage, modification, or deletion of data.
- Examples:
 - Simple SQL Injection: ' ; DROP TABLE users; --
 - Blind SQL Injection: 1 AND 1=1
- Prevention:
 - Use parameterized queries or prepared statements.
 - Validate and sanitize user inputs.
 - Implement least privilege access to the database.

2. Cross-Site Scripting (XSS):

- Description: XSS occurs when an attacker injects malicious scripts into a web page that other users can view. This can lead to session hijacking, defacement, or redirection to malicious websites.
- Types:
 - Stored XSS: Malicious script is stored on the server.
 - Reflected XSS: Malicious script is reflected off a web server.
 - DOM-based XSS: The vulnerability exists in the client-side code.

- Prevention:
 - Properly escape and validate all user inputs.
 - Use Content Security Policy (CSP).
 - Implement output encoding.

3. Cross-Site Request Forgery (CSRF):

- Description: CSRF tricks a user into performing actions on a web application without their knowledge. This can lead to unauthorized actions, such as changing passwords or making purchases.
- Prevention:
 - Implement anti-CSRF tokens.
 - Use same-site cookies.
 - Require re-authentication for sensitive actions.

4. Security Misconfiguration:

- Description: Security misconfiguration occurs when security settings are not properly defined or implemented. This can lead to exposure of sensitive information or unauthorized access.
- Common Issues:
 - Using default credentials.
 - Unnecessary features enabled.
 - Improper error handling.
- Prevention:
 - Regularly review and update configurations.
 - Disable unnecessary features and services.
 - Implement proper error handling and logging.

5. Broken Authentication & Session Management:

- Description: This vulnerability arises from weak authentication mechanisms, session fixation, or improper session management. It can lead to unauthorized access.
- Prevention:
 - Implement multi-factor authentication (MFA).
 - Use secure session identifiers and cookies.

- Enforce strong password policies and secure storage.

6. Sensitive Data Exposure:

- Description: Sensitive data exposure occurs when sensitive information, such as credit card numbers or personal data, is not properly protected.
- Prevention:
 - Use strong encryption for data at rest and in transit.
 - Implement access controls to restrict data access.
 - Avoid storing sensitive data unless absolutely necessary.

7. Insecure Deserialization:

- Description: Insecure deserialization happens when untrusted data is used to recreate objects, leading to remote code execution or other attacks.
- Prevention:
 - Validate and sanitize all inputs before deserialization.
 - Use safe serialization libraries and formats.
 - Implement integrity checks for serialized data.

8. Using Components with Known Vulnerabilities:

- Description: Using outdated or vulnerable software libraries and components can expose applications to attacks.
- Prevention:
 - Regularly update and patch software and libraries.
 - Use software composition analysis tools to identify vulnerabilities.
 - Monitor vulnerability databases for updates.

9. Insufficient Logging & Monitoring:

- Description: Lack of proper logging and monitoring can make it difficult to detect and respond to security incidents.
- Prevention:
 - Implement comprehensive logging for all critical actions and events.
 - Regularly review and analyze logs for suspicious activity.
 - Use monitoring tools to detect and alert on potential security incidents.

10. XML External Entity (XXE):

- Description: XXE occurs when XML input containing a reference to an external entity is processed by a weakly configured XML parser. It can lead to data exposure, denial of service, or server-side request forgery.
- Prevention:
 - Disable external entity resolution in XML parsers.
 - Use less complex data formats such as JSON.
 - Implement proper input validation and output encoding.

Task 2: Some of the website security measures are :

1. HTTPS (Hypertext Transfer Protocol Secure):

- Description: HTTPS encrypts data transmitted between a user's browser and the website, ensuring that sensitive information like login credentials, payment details, and personal data is secure from eavesdroppers.
- Implementation: Obtain an SSL/TLS certificate from a trusted certificate authority (CA) and configure your web server to use HTTPS.

2. Content Security Policy (CSP):

- Description: CSP is a security standard that helps prevent cross-site scripting (XSS) and other code injection attacks by specifying which sources are allowed to load content on a web page.
- Implementation: Define a CSP policy in the website's HTTP headers to control resources such as scripts, stylesheets, and images.

3. CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart):

- Description: CAPTCHA is a challenge-response test used to determine whether the user is human or a bot. It helps prevent automated attacks such as brute-force login attempts and spam.
- Implementation: Integrate CAPTCHA challenges (e.g., image recognition, math problems) on login forms, comment sections, and other areas susceptible to automated abuse.

4. Firewalls:

- Description: Firewalls monitor and control incoming and outgoing network traffic based on predetermined security rules. They help block unauthorized access and protect against various attacks.
- Implementation: Use web application firewalls (WAFs) to filter and block malicious traffic targeting your web applications.

5. Regular Software Updates:

- Description: Keeping software, plugins, and frameworks up to date ensures that known vulnerabilities are patched, reducing the risk of exploitation.
- Implementation: Regularly update your content management system (CMS), plugins, themes, and other software components.

6. Secure Coding Practices:

- Description: Writing secure code helps prevent vulnerabilities such as SQL injection, XSS, and CSRF. Following secure coding guidelines and best practices is essential.
- Implementation: Use input validation, output encoding, parameterized queries, and other secure coding techniques.

7. Access Control:

- Description: Implementing access control measures ensures that only authorized users can access sensitive data and functionalities.
- Implementation: Use strong authentication methods (e.g., multi-factor authentication), role-based access control, and least privilege principles.

8. Data Encryption:

- Description: Encrypting sensitive data at rest and in transit protects it from unauthorized access and theft.
- Implementation: Use strong encryption algorithms to encrypt data stored in databases and transmitted over the network.

9. Backup and Recovery Plans:

- Description: Regularly backing up website data and having a recovery plan in place ensures that you can restore your site in case of a security breach or data loss.
- Implementation: Schedule regular backups and store them in secure, offsite locations. Test your recovery procedures periodically.

10. Monitoring and Logging:

- Description: Continuous monitoring and logging of website activity help detect and respond to security incidents promptly.
- Implementation: Use monitoring tools to track access logs, error logs, and other relevant data. Set up alerts for suspicious activities.